

Dukan AI

Shop Dashboard – Source Code

File: `src/routes/index.tsx`

Lines: 555

```

1  import { createFileRoute } from "@tanstack/react-router";
2  import { useEffect, useMemo, useState } from "react";
3  import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
4  import { Button } from "@components/ui/button";
5  import { Input } from "@components/ui/input";
6  import { Label } from "@components/ui/label";
7  import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
8  import {
9    Table, TableBody, TableCell, TableHead, TableHeader, TableRow,
10 } from "@components/ui/table";
11 import { Badge } from "@components/ui/badge";
12 import {
13   Trash2, Plus, Store, TrendingUp, Package, IndianRupee, AlertTriangle,
14   Search, BarChart3, X
15 } from "lucide-react";
16 import { toast, Toaster } from "sonner";
17 import {
18   BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer, CartesianGrid,
19 } from "recharts";
20
21 export const Route = createFileRoute("/")({
22   head: () => ({
23     meta: [
24       { title: "Dukan AI – Shop Dashboard" },
25       { name: "description", content: "Manage stock, calculate sales and track profits for your shop." },
26       { property: "og:title", content: "Dukan AI – Shop Dashboard" },
27       { property: "og:description", content: "Manage stock, calculate sales and track profits for your shop." },
28     ],
29   }),
30   component: Index,
31 });
32
33 type Product = {
34   id: string;
35   name: string;
36   costPrice: number;
37   sellPrice: number;
38   stock: number;
39 };
40
41 type Sale = {
42   id: string;
43   productId: string;
44   productName: string;
45   qty: number;
46   revenue: number;
47   profit: number;
48   at: number;
49 };
50
51 const PRODUCTS_KEY = "dukan.products";
52 const SALES_KEY = "dukan.sales";
53
54 const seedProducts: Product[] = [
55   { id: "p1", name: "Parle-G Biscuit", costPrice: 8, sellPrice: 10, stock: 50 },
56   { id: "p2", name: "Amul Milk 1L", costPrice: 58, sellPrice: 66, stock: 24 },
57   { id: "p3", name: "Maggi 70g", costPrice: 12, sellPrice: 14, stock: 40 },
58   { id: "p4", name: "Red Label Tea 250g", costPrice: 140, sellPrice: 160, stock: 12 },
59   { id: "p5", name: "Colgate 200g", costPrice: 95, sellPrice: 110, stock: 8 },
60 ];
61
62 function load<T>(key: string, fallback: T): T {
63   if (typeof window === "undefined") return fallback;
64   try {
65     const raw = localStorage.getItem(key);
66     return raw ? (JSON.parse(raw) as T) : fallback;
67   } catch {
68     return fallback;
69   }

```

```

81  const [products, setProducts] = useState<Product[]>([]);
82  const [sales, setSales] = useState<Sale[]>([]);
83  const [hydrated, setHydrated] = useState(false);
84
85  useEffect(() => {
86    setProducts(load(PRODUCTS_KEY, seedProducts));
87    setSales(load(SALES_KEY, []));
88    setHydrated(true);
89  }, []);
90
91  useEffect(() => {
92    if (hydrated) localStorage.setItem(PRODUCTS_KEY, JSON.stringify(products));
93  }, [products, hydrated]);
94  useEffect(() => {
95    if (hydrated) localStorage.setItem(SALES_KEY, JSON.stringify(sales));
96  }, [sales, hydrated]);
97
98  const totals = useMemo(() => {
99    const revenue = sales.reduce((s, x) => s + x.revenue, 0);
100   const profit = sales.reduce((s, x) => s + x.profit, 0);
101   const stockValue = products.reduce((s, p) => s + p.costPrice * p.stock, 0);
102   const lowStock = products.filter((p) => p.stock <= 5).length;
103   return { revenue, profit, stockValue, lowStock, orders: sales.length };
104 }, [sales, products]);
105
106  const chartData = useMemo(() => {
107    const byDay = new Map<string, number>();
108    sales.forEach((s) => {
109      const day = new Date(s.at).toLocaleDateString("en-IN");
110      byDay.set(day, (byDay.get(day) || 0) + s.profit);
111    });
112    const sorted = Array.from(byDay.entries()).sort((a, b) => {
113      const ad = new Date(a[0].split("/").reverse().join("-"));
114      const bd = new Date(b[0].split("/").reverse().join("-"));
115      return ad.getTime() - bd.getTime();
116    });
117    return sorted.slice(-7).map(([name, profit]) => ({ name: name.split("/").slice(0, 2).join("/"), profit }));
118  }, [sales]);
119
120  return (
121    <div className="min-h-screen bg-background text-foreground">
122      <Toaster richColors position="top-center" />
123      <header className="sticky top-0 z-10 border-b bg-card/80 backdrop-blur">
124        <div className="mx-auto flex max-w-6xl items-center justify-between px-4 py-3">
125          <div className="flex items-center gap-3">
126            <div className=
127"flex h-9 w-9 items-center justify-center rounded-lg bg-primary text-primary-foreground shadow-sm">
128              <Store className="h-4 w-4" />
129            </div>
130            <div>
131              <h1 className="text-base font-bold leading-tight">Dukan AI</h1>
132              <p className="text-[11px] text-muted-foreground">Smart shop dashboard</p>
133            </div>
134          </div>
135          <Badge variant="secondary" className="hidden sm:inline-flex text-xs">v1.0</Badge>
136        </div>
137      </header>
138      <main className="mx-auto max-w-6xl px-3 sm:px-4 py-4 sm:py-6 space-y-5">
139        <section className="grid gap-3 sm:gap-4 grid-cols-2 lg:grid-cols-4">
140          <StatCard icon={}<IndianRupee className="h-4 w-4" /> label="Total Sales" value={inr(totals.revenue)} />
141          <StatCard icon={}<TrendingUp className="h-4 w-4" /> label="Total Profit" value={inr(totals.profit)} accent
142/>
143          <StatCard icon={}<Package className="h-4 w-4" /> label="Stock Value" value={inr(totals.stockValue)} />
144          <StatCard icon={}<AlertTriangle className="h-4 w-4" /> label="Low Stock" value={String(totals.lowStock)} />
145        </section>
146        {chartData.length > 0 && (

```

```

161         <ResponsiveContainer width="100%" height="100%">
162             <BarChart data={chartData}>
163                 <CartesianGrid strokeDasharray="3 3" stroke="var(--border)" />
164                 <XAxis dataKey="name" tick={{ fontSize: 11 }} />
165                 <YAxis tickFormatter={(v) => `₹${v}`} tick={{ fontSize: 11 }} width={50} />
166                 <Tooltip formatter={(v: number) => inr(v)} />
167                 <Bar dataKey="profit" fill="var(--primary)" radius={[4, 4, 0, 0]} />
168             </BarChart>
169         </ResponsiveContainer>
170     </div>
171 </CardContent>
172 </Card>
173 }}
174
175 <Tabs defaultValue="sell" className="w-full">
176     <TabsList className="grid w-full grid-cols-3 h-11">
177         <TabsTrigger value="sell" className="text-sm">Sell</TabsTrigger>
178         <TabsTrigger value="stock" className="text-sm">Stock</TabsTrigger>
179         <TabsTrigger value="history" className="text-sm">History</TabsTrigger>
180     </TabsList>
181
182     <TabsContent value="sell" className="mt-3">
183         <SellCalculator
184             products={products}
185             onSell={(productId, qty) => {
186                 const p = products.find((x) => x.id === productId);
187                 if (!p) return;
188                 if (qty <= 0) return toast.error("Quantity must be > 0");
189                 if (qty > p.stock) return toast.error(`Only ${p.stock} in stock`);
190                 const revenue = p.sellPrice * qty;
191                 const profit = (p.sellPrice - p.costPrice) * qty;
192                 setProducts((prev) =>
193                     prev.map((x) => (x.id === p.id ? { ...x, stock: x.stock - qty } : x)),
194                 );
195                 setSales((prev) => [
196                     { id: crypto.randomUUID(), productId: p.id, productName: p.name, qty, revenue, profit, at: Date.now
197                 () },
198                     ...prev,
199                 ]);
200                 toast.success(`Sold ${qty} × ${p.name} – profit ${inr(profit)}`);
201             }}
202         />
203     </TabsContent>
204
205     <TabsContent value="stock" className="mt-3">
206         <StockManager
207             products={products}
208             onAdd={(p) => setProducts((prev) => [...prev, p])}
209             onRemove={(id) => setProducts((prev) => prev.filter((x) => x.id !== id))}
210             onRestock={(id, qty) =>
211                 setProducts((prev) => prev.map((x) => (x.id === id ? { ...x, stock: x.stock + qty } : x)))
212             }
213         />
214     </TabsContent>
215
216     <TabsContent value="history" className="mt-3">
217         <SalesHistory sales={sales} onClear={() => setSales([])} />
218     </TabsContent>
219 </Tabs>
220 </main>
221 </div>
222 );
223 }
224
225 function
    StatCard({ icon, label, value, accent }: { icon: React.ReactNode; label: string; value: string; accent?: boolean })
    {
226     return (

```

```

241 }
242
243 function SellCalculator({ products, onSell }: { products: Product[]; onSell: (id: string, qty: number) => void }) {
244   const [productId, setProductId] = useState<string>("");
245   const [qty, setQty] = useState<number>(1);
246   const p = products.find((x) => x.id === productId);
247   const revenue = p ? p.sellPrice * qty : 0;
248   const profit = p ? (p.sellPrice - p.costPrice) * qty : 0;
249
250   return (
251     <Card>
252       <CardHeader className="pb-2">
253         <CardTitle className="text-sm sm:text-base">Quick Sale Calculator</CardTitle>
254       </CardHeader>
255       <CardContent className="space-y-4">
256         {products.length === 0 ? (
257           <p className="text-sm text-muted-foreground">Add products in the Stock tab first.</p>
258         ) : (
259           <>
260             <div className="grid gap-3">
261               <div className="space-y-1.5">
262                 <Label className="text-xs sm:text-sm">Product</Label>
263                 <select
264                   value={productId}
265                   onChange={(e) => setProductId(e.target.value)}
266                   className="flex h-11 w-full rounded-md border bg-background px-3 text-sm"
267                 >
268                   <option value="">Select product...</option>
269                   {products.map((x) => (
270                     <option key={x.id} value={x.id}>
271                       {x.name} – {inr(x.sellPrice)} ({x.stock} left)
272                     </option>
273                   ))}
274                 </select>
275               </div>
276               <div className="space-y-1.5">
277                 <Label className="text-xs sm:text-sm">Quantity</Label>
278                 <div className="flex items-center gap-2">
279                   <button
280                     className="h-11 w-11 rounded-md border bg-muted/40 text-lg font-bold active:bg-muted"
281                     onClick={() => setQty((q) => Math.max(1, q - 1))}
282                     type="button"
283                   >
284                     –
285                   </button>
286                   <Input
287                     type="number"
288                     min={1}
289                     value={qty}
290                     onChange={(e) => setQty(Math.max(1, Number(e.target.value) || 1))}
291                     className="text-center text-lg font-semibold h-11"
292                   />
293                   <button
294                     className="h-11 w-11 rounded-md border bg-muted/40 text-lg font-bold active:bg-muted"
295                     onClick={() => setQty((q) => q + 1)}
296                     type="button"
297                   >
298                     +
299                   </button>
300                 </div>
301               </div>
302             </div>
303
304             <div className="grid gap-2 grid-cols-3">
305               <Mini label="Revenue" value={inr(revenue)} />
306               <Mini label="Profit" value={inr(profit)} highlight />
307               <Mini label="Left after" value={p ? String(Math.max(0, p.stock - qty)) : "-"} />
308             </div>

```

```

321         setQty(1);
322     }}
323     >
324         Record Sale
325     </Button>
326 </>
327     )}
328 </CardContent>
329 </Card>
330 );
331 }
332
333 function Mini({ label, value, highlight }: { label: string; value: string; highlight?: boolean }) {
334     return (
335         <div className={
336             `rounded-lg border p-2 sm:p-3 text-center ${highlight ? "bg-accent/50 border-primary/30" : "bg-muted/30"}`>
337             <div className="text-[10px] sm:text-xs text-muted-foreground">{label}</div>
338             <div className="text-sm sm:text-lg font-semibold tabular-nums">{value}</div>
339         </div>
340     );
341 }
342
343 function StockManager({
344     products, onAdd, onRemove, onRestock,
345 }: {
346     products: Product[];
347     onAdd: (p: Product) => void;
348     onRemove: (id: string) => void;
349     onRestock: (id: string, qty: number) => void;
350 }) {
351     const [name, setName] = useState("");
352     const [cost, setCost] = useState<number>(0);
353     const [sell, setSell] = useState<number>(0);
354     const [stock, setStock] = useState<number>(0);
355     const [search, setSearch] = useState("");
356     const filtered = useMemo(() => {
357         const q = search.trim().toLowerCase();
358         return q ? products.filter((p) => p.name.toLowerCase().includes(q)) : products;
359     }, [products, search]);
360
361     return (
362         <div className="space-y-4">
363             <Card>
364                 <CardHeader className="pb-2">
365                     <CardTitle className="text-sm sm:text-base">Add Product</CardTitle>
366                 </CardHeader>
367                 <CardContent className="space-y-3">
368                     <div className="space-y-1.5">
369                         <Label className="text-xs sm:text-sm">Name</Label>
370                         <Input value={name} onChange={(e) => setName(e.target.value)} placeholder="e.g. Tata Salt 1kg" />
371                     </div>
372                     <div className="grid grid-cols-2 gap-3">
373                         <div className="space-y-1.5">
374                             <Label className="text-xs sm:text-sm">Cost ■</Label>
375                             <Input type="number" min={0} value={cost} onChange={(e) => setCost(Number(e.target.value) || 0)} />
376                         </div>
377                         <div className="space-y-1.5">
378                             <Label className="text-xs sm:text-sm">Sell ■</Label>
379                             <Input type="number" min={0} value={sell} onChange={(e) => setSell(Number(e.target.value) || 0)} />
380                         </div>
381                     </div>
382                     <div className="space-y-1.5">
383                         <Label className="text-xs sm:text-sm">Initial Stock</Label>
384                         <Input type="number" min={0} value={stock} onChange={(e) => setStock(Number(e.target.value) || 0)} />
385                     </div>
386                     <Button
387                         className="w-full"
388                         onClick={() => {

```

```

401     </Button>
402   </CardContent>
403 </Card>
404
405 <Card>
406   <CardHeader className="pb-2 flex flex-row items-center justify-between gap-2">
407     <CardTitle className="text-sm sm:text-base">Inventory</CardTitle>
408     <Badge variant="secondary">{filtered.length}</Badge>
409   </CardHeader>
410   <CardContent className="space-y-3">
411     <div className="relative">
412       <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" />
413       <Input
414         value={search}
415         onChange={(e) => setSearch(e.target.value)}
416         placeholder="Search products..."
417         className="pl-9"
418       />
419       {search && (
420         <button
421           className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground"
422           onClick={() => setSearch("")}
423         >
424           <X className="h-4 w-4" />
425         </button>
426       )}
427     </div>
428
429     {filtered.length === 0 ? (
430       <p className="text-sm text-muted-foreground text-center py-6">No products found.</p>
431     ) : (
432       <div className="space-y-2">
433         {/* Desktop table */}
434         <div className="hidden sm:block overflow-x-auto">
435           <Table>
436             <TableHeader>
437               <TableRow>
438                 <TableHead>Product</TableHead>
439                 <TableHead className="text-right">Cost</TableHead>
440                 <TableHead className="text-right">Sell</TableHead>
441                 <TableHead className="text-right">Margin</TableHead>
442                 <TableHead className="text-right">Stock</TableHead>
443                 <TableHead className="text-right">Actions</TableHead>
444             </TableRow>
445           </TableHeader>
446           <TableBody>
447             {filtered.map((p) => (
448               <TableRow key={p.id}>
449                 <TableCell className="font-medium">{p.name}</TableCell>
450                 <TableCell className="text-right tabular-nums">{inr(p.costPrice)}</TableCell>
451                 <TableCell className="text-right tabular-nums">{inr(p.sellPrice)}</TableCell>
452                 <TableCell className="text-right tabular-nums">{inr(p.sellPrice - p.costPrice)}</TableCell>
453                 <TableCell className="text-right">
454                   {p.stock <= 5 ? (
455                     <Badge variant="destructive">{p.stock}</Badge>
456                   ) : (
457                     <span className="tabular-nums">{p.stock}</span>
458                   )}
459                 </TableCell>
460                 <TableCell className="text-right">
461                   <div className="flex justify-end gap-1">
462                     <Button size="sm" variant="outline" onClick={() => onRestock(p.id, 10)}>+10</Button>
463                     <Button size="sm" variant="ghost" onClick={() => onRemove(p.id)}>
464                       <Trash2 className="h-4 w-4" />
465                     </Button>
466                   </div>
467                 </TableCell>
468               </TableRow>

```

```

481         {filtered.map((p) => (
482             <div key={p.id} className="rounded-lg border p-3 space-y-2 bg-card">
483                 <div className="flex items-center justify-between">
484                     <span className="font-medium text-sm">{p.name}</span>
485                     {p.stock <= 5 ? (
486                         <Badge variant="destructive" className="text-[10px]">Low: {p.stock}</Badge>
487                     ) : (
488                         <Badge variant="secondary" className="text-[10px]">Stock: {p.stock}</Badge>
489                     )}
490                 </div>
491                 <div className="grid grid-cols-3 gap-2 text-xs text-muted-foreground">
492                     <div>Cost: <span className="text-foreground font-medium">{inr(p.costPrice)}</span></div>
493                     <div>Sell: <span className="text-foreground font-medium">{inr(p.sellPrice)}</span></div>
494                     <div>Margin: <span className="text-foreground font-medium"
495 >{inr(p.sellPrice - p.costPrice)}</span></div>
496                 </div>
497                 <div className="flex gap-2 pt-1">
498                     <Button size="sm" variant="outline" className="flex-1" onClick={() => onRestock(p.id, 10)}>Rest
499 ock +10</Button>
500                     <Button size="sm" variant="ghost" className="px-3" onClick={() => onRemove(p.id)}>
501                         <Trash2 className="h-4 w-4" />
502                     </Button>
503                 </div>
504             </div>
505         </div>
506     )}
507 </CardContent>
508 </Card>
509 </div>
510 );
511 }
512
513 function SalesHistory({ sales, onClear }: { sales: Sale[]; onClear: () => void }) {
514     return (
515         <Card>
516             <CardHeader className="flex flex-row items-center justify-between pb-2">
517                 <CardTitle className="text-sm sm:text-base">Recent Sales ({sales.length})</CardTitle>
518                 {sales.length > 0 && (
519                     <Button variant="ghost" size="sm" onClick={onClear}>Clear</Button>
520                 )}
521             </CardHeader>
522             <CardContent>
523                 {sales.length === 0 ? (
524                     <p className="text-sm text-muted-foreground text-center py-8">No sales yet. Use the Sell tab to record one.
525 </p>
526                 ) : (
527                     <div className="space-y-2">
528                         {/* Desktop table */}
529                         <div className="hidden sm:block overflow-x-auto">
530                             <Table>
531                                 <TableHeader>
532                                     <TableRow>
533                                         <TableHead>Time</TableHead>
534                                         <TableHead>Product</TableHead>
535                                         <TableHead className="text-right">Qty</TableHead>
536                                         <TableHead className="text-right">Revenue</TableHead>
537                                         <TableHead className="text-right">Profit</TableHead>
538                                     </TableRow>
539                                 </TableHeader>
540                                 <TableBody>
541                                     {sales.map((s) => (
542                                         <TableRow key={s.id}>
543                                             <TableCell className="text-muted-foreground text-xs">
544                                                 {new Date(s.at).toLocaleString()}
545                                             </TableCell>
546                                             <TableCell className="font-medium">{s.productName}</TableCell>

```



```

561     <div className="sm:hidden space-y-2">
562       {sales.map((s) => (
563         <div key={s.id} className="rounded-lg border p-3 bg-card">
564           <div className="flex items-center justify-between">
565             <span className="font-medium text-sm">{s.productName}</span>
566             <span className="text-xs text-muted-foreground">{new Date(s.at).toLocaleDateString()}</span>
567           </div>
568           <div className="grid grid-cols-3 gap-2 mt-2 text-xs">
569             <div className="text-center rounded bg-muted/30 p-2">
570               <div className="text-muted-foreground">Qty</div>
571               <div className="font-semibold">{s.qty}</div>
572             </div>
573             <div className="text-center rounded bg-muted/30 p-2">
574               <div className="text-muted-foreground">Revenue</div>
575               <div className="font-semibold">{inr(s.revenue)}</div>
576             </div>
577             <div className="text-center rounded bg-accent/40 p-2 border border-primary/20">
578               <div className="text-muted-foreground">Profit</div>
579               <div className="font-semibold text-primary">{inr(s.profit)}</div>
580             </div>
581           </div>
582         </div>
583       )})
584     </div>
585   </div>
586   </div>
587 </CardContent>
588 </Card>
589 );
590 }
591

```